CIS1300 Assignment 3 — MiniNLP: Trend Analyzer for Short Posts

- This assignment is worth 4% of your final grade.

Learning Outcomes

- Use loops and **nested loops** to solve problems step by step.
- Create and use **user-defined functions** to break a problem into smaller parts.
- Work with **one and two dimensional arrays and strings** to store and process data.
- Read requirements carefully and translate them into working C code.

1.0 NLP – Natural Language Processing

Natural Language Processing, or NLP, is a field of computer science that helps computers understand and work with human language. Human language is messy: people use punctuation, slang, abbreviations, different word forms, spelling variations, and informal expressions. NLP gives computers ways to clean, organize, and analyze text so they can do useful tasks such as detecting spam, finding common topics in student feedback, translating languages, summarizing reviews, or powering chatbots. In simple terms, NLP is about teaching computers how to make sense of words written or spoken by people. Some common terms used in NLP and those that you need to understand for this assignment are:

- **Stop words** are common words like *"the", "and", "to"* that usually don't carry meaning for keyword trends, so we remove them.
- **Stemming** chops off endings like *-ing, -ed, -s* to reduce words to a simpler form (e.g., *errors → error*).
- **Lemmatization** converts different forms of a word into a "dictionary form" (e.g., *studied/studying → study*). In this assignment, you'll do it using a small lookup table provided as input.

2.0 Rules / Restrictions

- No structs are allowed
- Must compile with: `-std=c99 -Wall` (no warnings).
- Must use:
  - the required user-defined functions described below
  - DO NOT change any prototype given in `givenA3.h` – your program will not compile otherwise

3.0 Input

Your program reads three things as shown below. These are done using function **readFile**, the function description for which is given to you. You must call this function in main before you write the other required functions.

3.1 Stop word list: Stored in:

- `char stop[MAX_STOP][MAX_WORD];`

3.2 Lemma map (mini lemmatization table): Stored in:

- `char variant[MAX_LEMMA][MAX_WORD];`
- `char lemma[MAX_LEMMA][MAX_WORD];`

3.3 Posts: Stored in:

- `char posts[MAX_POSTS][MAX_LEN];`

4.0 Output

See sample run1 and sample run2 given in section 5.3.

5.0 Your Task: complete both parts A and B

First things first

You are given a header file called givenA3.h – this file contains the proper function prototypes, constants, for this assignment. Note that this file has an extension of h.

You are also given 2 incomplete source files:
a. givenA3.c – this file has the function definition of readFile.
b. givenA3Main.c – this file contains main() that calls readFile to populate the initial arrays and values required for other tasks.

Important: note the first line in givenA3.c and givenA3Main.c (#include "givenA3.h"). Do not remove it.

You must submit the definitions and calls to all required functions given in section 5:

(1) Rename givenA3.c as lastnameFirstnameA3.c – add to it, the function implementations of each task detailed below in section 5.1
(2) Rename givenA3Main.c as lastnameFirstnameA3Main.c – add to it, function calls to each task - as described in section 5.2

To compile them together, use the following gcc command:

```
gcc -std=c99 -Wall lastnameFirstnameA3.c lastnameFirstnameA3Main.c

or

gcc -std=c99 -Wall lastnameFirstnameA3.c lastnameFirstnameA3Main.c -o assn3
```

More on this can be found in the section on "Submission Instructions".

You are also free to create additional helper functions, though if they are used by the assignment functions below, include them in the appropriate files so they are accessible to us during grading.

5.1 Part A: Write function definitions for the following tasks.

Task 1

```
void normalize(char line[]);
```

This function takes as input one string called `line`. It modifies `line` directly by normalizing the text. Normalizing means that the function must:

- convert all uppercase alphabetic characters to lowercase
- replace punctuation characters with spaces

For this assignment, the following punctuation characters must be replaced with spaces:

```
. , ! ? : ; " ( ) [ ]
```

Digits and alphabetic characters should remain unchanged.

Example:

```
Original line:  I LOVE coding, debugging, and tests!!!
After normalize: i love coding  debugging  and tests
```

Note that this function modifies the original string and does not return any value.

Task 2

```
int tokenize(const char line[], char tokens[][MAX_WORD]);
```

This function takes as input a normalized string called `line` and a 2-dimensional character array called `tokens`.

The function splits `line` into individual words, called tokens, and stores each token in the `tokens` array.

Tokens are separated by spaces. The function must ignore repeated spaces. For example, if the input line contains several spaces between two words, the function should not create empty tokens.

Example:

```
Input line:      i   love   coding
Stored tokens:   i
                 love
                 coding
```

The function returns the total number of tokens stored in the `tokens` array. Note that this function does not print anything.

Task 3

```
int removeStopWords(char tokens[][MAX_WORD], int numToken,
                    char stop[][MAX_WORD], int numStop);
```

This function takes as input:

- the 2-dimensional character array `tokens`, which stores the words from a post
- the integer `numToken`, which represents the number of tokens currently stored
- the 2-dimensional character array `stop`, which stores the stop words
- the integer `numStop`, which represents the number of stop words stored in `stop`

The function removes all stop words from `tokens`.

A stop word is a common word such as `the`, `a`, `to`, `and`, or `is` that is usually removed before analyzing text.

If a token matches any stop word exactly, that token must be removed from the list of tokens.

Example:

```
Tokens before removing stop words:
i love coding and debugging

Stop words:
i and

Tokens after removing stop words:
love coding debugging
```

The function returns the new number of tokens after stop words have been removed. Note that this function does not print anything.

Task 4

```
void applyLemmaMap(char tokens[][MAX_WORD],
                   int count,
                   char variant[][MAX_WORD],
                   char lemma[][MAX_WORD],
                   int L);
```

This function takes as input:

- the 2-dimensional character array `tokens`
- the integer `count`, which represents the number of tokens
- the 2-dimensional character array `variant`, which stores word variants
- the 2-dimensional character array `lemma`, which stores the base word for each variant
- the integer `L`, which represents the number of lemma pairs

The function performs simple lemmatization using a lookup table.

Lemmatization means converting a word to its base or dictionary form. In this assignment, lemmatization is done using the `variant` and `lemma` arrays.

For each token, if the token matches `variant[i]`, it must be replaced with `lemma[i]`.

Example:

```
variant[0] = studying     lemma[0] = study
variant[1] = studied      lemma[1] = study
variant[2] = went         lemma[2] = go
```

If the tokens are:

```
studying went home
```

After applying the lemma map, the tokens become:

```
study go home
```

The function modifies the `tokens` array directly and does not return any value. Note that this function does not print anything.

5.2 Part B: Main Program

You must write a main program that calls the functions in this order and displays an output as shown in Sample runs 1 and 2 below.

The main program must (in this order):

1. Call `readFile` to populate the required arrays (stop, variant, lemma and post) and ints (numPosts, numStop and numLemma) – note that you are given the function description of `readFile`
2. For each post read from the file: call the following functions in this order:
    1. `normalize`
    2. `tokenize`
    3. `removeStopWords`
    4. `applyLemmaMap`

5.3 Sample Runs

Sample Run 1

Enter filename: file0.input

Post 1 original: We studied pointers and debugging; "debugging" is hard.

Number of tokens = 8

Num of stop words removed = 3

Post 1 cleaned: study pointers debugging debugging hard

Sample Run 2

Enter filename: file2.input

Post 1 original: Went to the gym, then studied!

Number of tokens = 6

Num of stop words removed = 2

Post 1 cleaned: go gym then study

Post 2 original: Studies in CIS1300 are intense; errors happen.

Number of tokens = 7

Num of stop words removed = 0

Post 2 cleaned: study in cis1300 are intense errors happen

Post 3 original: Going to fix fixes fixing!!!

Number of tokens = 5

Num of stop words removed = 1

Post 3 cleaned: go fix fixes fixing

Post 4 original: Better sleep is good for coding and debugging.

Number of tokens = 8

Num of stop words removed = 3

Post 4 cleaned: good sleep good coding debugging

6 Program Submission

You must submit 2 files lastnameFirstnameA3.c and lastnameFirstnameA3Main.c
Incorrect file names will result in penalty. DO NOT submit givenA3.h

6.1 Program Expectations

Your program is expected to follow the outlined information exactly. Failure to do so will result in deductions to your assignment grade.

- The program files you submit MUST compile with NO warnings and errors using the flags - std=c99 and -Wall.
- If you decide to define and use helper functions (other than what is listed in the .h file), then you must also add their prototypes in lastnameFirstnameA3.c, along with their definitions.
- **Programs that fail to compile will be given a mark of 0.**
- Programs that produce warnings upon compilation will receive a deduction of 1 mark for each type/category of warning.
- Penalties will occur for missing style, comments, header comments.
- Both program files must contain the header comment shown in section 6.4 with the underlined & bolded sections properly filled in.

6.2 List of "Do Not"s

- Do not change any given function prototype or constant values
- Do not use Global Variables
    - Using global variables will result in 0 in the assignment
- Do not use GOTO statements
    - Using Goto statements will result in 0 in the assignment

6.3 Academic Integrity and Use of AI

You may use course materials, notes, and the textbook to support your learning.

Generative AI tools (e.g., ChatGPT, Copilot, Claude) may be used as learning supports, including to help explain concepts, review terminology, or discuss approaches to improving requirement quality.

However:

- You cannot use AI tools to complete this assignment.
- You remain responsible for all submitted work.
- Your final submission must demonstrate your own understanding and critical thinking.

6.4 Header Comment

Use the templates given below for header comment – you must include them in both c files you submit.
/!\ Note: The file name, student name and email ID must be changed per file and per student.

```
/************************chaturvediRituA3.c**************
Student Name: Ritu Chaturvedi Email Id: ritu

   Due Date: November … Course Name: CIS 1300

   I have exclusive control over this submission via my password.
   By including this statement in this header comment, I certify that:

   1) I have read and understood the University policy on academic integrity. 2) I
   have completed the Computing with Integrity Tutorial on Moodle; and 3) I have
   achieved at least 80% in the Computing with Integrity Self Test.

   I assert that this work is my own. I have appropriately acknowledged any and
   all material that I have used, whether directly quoted or paraphrased.
   Furthermore, I certify that this assignment was prepared by me specifically for
   this course.

*********************************************************/
```

The program file must contain instructions for the TA on how to compile and run your program in a header comment.

/!\ Note: The file name must be changed per student.

```
/*********************************************************
Compiling the program
The program should be compiled using the following flags: -std=c99 -Wall

compiling:
gcc -std=c99 -Wall chaturvediRituA3.c chaturvediRituA3Main.c
```

```
Running: ./a.out


OR

gcc -std=c99 -Wall chaturvediRituA3.c chaturvediRituA3Main.c -o assn3

Running the Program: ./assn3
*********************************************************/
```